
PROGRAMMATION CONCURRENTE EN MULTI-THREADS

Livret de travaux dirigés

2026/2027

Matthieu PERRIN
Nantes Université
Laboratoire des Sciences du Numérique de Nantes
UMR CNRS 6004
Bureau 410, bâtiment 34
matthieu.perrin@univ-nantes.fr

Séance 1

Du parallélisme à la concurrence

Exercice 1.1 (La loi d'Amdahl).

Une proportion $p = 70\%$ d'un programme séquentiel peut être parallélisée. On dispose d'une machine avec $n = 10$ cœurs.

1a. Quelle est l'accélération (speedup) attendue ?

$$S(10) = \frac{1}{(1-p) + \frac{p}{10}} = \frac{1}{0,3 + \frac{0,7}{10}} \approx 2,70.$$

1b. On trouve dans le commerce une machine avec $n = 20$ cœurs. Quelle est l'accélération attendue ?

$$S(20) = \frac{1}{(1-p) + \frac{p}{20}} = \frac{1}{0,3 + \frac{0,7}{20}} \approx 2,99.$$

1c. Quelle est l'accélération maximale théorique lorsque $n \rightarrow \infty$?

$$\lim_{n \rightarrow \infty} S(n) = \frac{1}{1-p} = \frac{1}{0,3} \approx 3,33.$$

Un ingénieur pense qu'il existe un moyen d'optimiser la partie séquentielle pour la rendre deux fois plus rapide.

1d. Quelle proportion du programme est parallélisable dans la version optimisée ?

On suppose maintenant que la partie séquentielle est deux fois plus rapide, mais que la partie non parallélisable est inchangée. On obtient donc :

$$T_1^{\text{opt}} = \frac{T_1 \times (1-p)}{2} + T_1 \times p.$$

La proportion parallélisable est donc :

$$p_{\text{opt}} = \frac{T_1 \times p}{T_1^{\text{opt}}} = \frac{2p}{1+p} \approx 82\%.$$

1e. Quelle accélération obtient-on alors, dans la parallélisation avec $n = 10$ cœurs ?

$$S_{\text{opt}}(10) = \frac{1}{(1-p_{\text{opt}}) + \frac{p_{\text{opt}}}{10}} \approx 3,86.$$

1f. À coût égal, vaut-il mieux choisir d'acheter la machine avec $n = 20$ cœurs ou payer l'ingénieur pour optimiser le code ?

Il faut comparer l'accélération par rapport à T_1 , de T_{20} , le temps d'exécution du programme initial sur la machine à 20 cœurs, et de T_{10}^{opt} , le temps d'exécution du programme optimisé sur la machine à 10 cœurs.

$$- \frac{T_{20}}{T_1} = S(20) \approx 2,99.$$

$$- \frac{T_{10}^{\text{opt}}}{T_1} = \frac{1-p}{2} + \frac{p}{10} \approx 4,55.$$

L'optimisation de la partie séquentielle est nettement plus rentable.

Séance 1

Du parallélisme à la concurrence

Exercice 1.2 (Parallélisation).

Le but de cet exercice est d'écrire une fonction `double[][] product(double[][] M1, double[][] M2)` qui calcule le produit de deux matrices carrées M_1 et M_2 de taille $n \times n$. On rappelle la définition de la multiplication des matrices. Si $M_3 = M_1 \times M_2$, alors pour tout $i, j \in \{0, \dots, n-1\}$,

$$M_3[i][j] = \sum_{k=0}^{n-1} M_1[i][k] \times M_2[k][j].$$

2a. Implémentez la version séquentielle de `product`.

```
public interface Product {
    double[][] product(double[][] M1, double[][] M2) throws InterruptedException;
}
public final class Sequential implements Product {
    double[][] product(double[][] M1, double[][] M2) {
        int n = M1.length;
        double[][] M3 = new double[n][n];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                M3[i][j] = 0;
                for (int k = 0; k < n; k++) {
                    M3[i][j] += M1[i][k] * M2[k][j];
                }
            }
        }
        return M3;
    }
}
```

2b. Proposez une parallélisation où chaque cellule $M_3[i][j]$ est calculée par un thread distinct (jusqu'à n^2 threads).

```
package tdl_parallelism.matrix;

public final class Naive implements Product {

    public double[][] product(double[][] M1, double[][] M2) throws InterruptedException {

        int n = M1.length;
        double[][] M3 = new double[n][n];
        Thread[] threads = new Thread[n*n];

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                final int ii = i, jj = j;
                threads[i*n+j] = new Thread(() -> {
                    M3[ii][jj] = 0;
                    for (int k = 0; k < n; k++) {
                        M3[ii][jj] += M1[ii][k] * M2[k][jj];
                    }
                });
            }
        }

        for (var t : threads) t.start();
    }
}
```

```

    for(var t : threads) t.join();

    return M3;
}
}

```

2c. Proposez une version qui crée au plus T threads (T est une variable globale), chacun calculant un lot de cellules. Précisez votre stratégie de répartition.

```

public static double[][] product(int n, double[][] M1, double[][] M2) {
    double[][] M = new double[n][n];
    Thread threads[][] = new Thread[n][n];
    for(int i = 0; i < n; i++) {
        int thisI = i;
        threads[i][0] = new Thread(() -> {
            for(int j = 0; j < n; j++) {
                int thisJ = j;
                threads[thisI][j] = new Thread(() -> {
                    M[thisI][thisJ] = 0;
                    for(int k = 0; k < n; k++) {
                        M[thisI][thisJ] += M1[thisI][k] * M2[k][thisJ];
                    }
                });
                threads[thisI][j].start();
            }
            for(int j = 1; j < n; j++) {
                try {
                    threads[thisI][j].join();
                } catch (InterruptedException e) { }
            }
        });
        threads[i][0].start();
    }
    for(int i = 1; i < n; i++) {
        try {
            threads[i][0].join();
        } catch (InterruptedException e) { }
    }
    return M;
}

```

2d. Analysez la complexité en temps des versions et justifiez l'intérêt de limiter le nombre de threads à une borne indépendante de n .

Si on parallélise la boucle intérieure, on passe d'une complexité $O(n^3)$ à $O(n^2)$, soit une accélération en $O(n)$. On peut faire mieux en parallélisant la boucle extérieure également. On passe alors de $O(n^3)$ à $O(n)$, soit une accélération en $O(n^2)$. Cela ne viole pas la loi d'Amdahl car on change le problème en changeant le nombre de threads : on augmente le nombre de threads pour s'adapter à la quantité de données à traiter. C'est l'idée de la loi de Gustafson : Si le nombre de threads est proportionnel à la taille des données, on peut espérer une accélération en $O(n)$.

Remarque : le programme ci-dessus utilise $n^2 + n + 1$ threads, mais on peut utiliser les mêmes threads pour paralléliser et pour faire certains calculs.

Séance 1

Du parallélisme à la concurrence

Exercice 1.3 (Asynchronie).

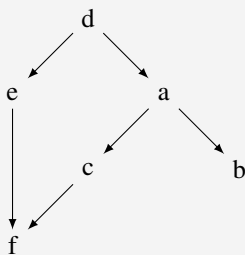
On se donne le programme multi-threads suivant :

```
Thread t = new Thread() -> {
    System.out.print("a");
    new Thread() -> { System.out.print("b"); }.start();
    System.out.print("c");
};

System.out.print("d");
t.start();
System.out.print("e");
t.join();
System.out.print("f");
```

- 3a.** Décrivez la relation “happens-before” du programme.
- 3b.** Décrivez toutes les sorties possibles pour le programme, en sachant qu’il est séquentiellement cohérent.
- 3c.** Décrivez la machine à états du programme.

Les sorties possibles sont toutes les linéarisations du graphe de causalité, soit les 11 sorties suivantes :



- deacfb
- deacbf
- deabcf
- daecfb
- daecbf
- daebcf
- dabecf
- dacefb
- dacebf
- dacbef
- dabcef

Séance 1 — pour aller plus loin

Du parallélisme à la concurrence

Exercice 1.4 (Le savant fou¹).

En visite pédagogique dans un laboratoire, vous et le reste de votre groupe êtes capturés par un savant fou. Il fait l'annonce suivante :

« J'ai mis en place une pièce qui contient une lumière et un interrupteur qui la commande. Actuellement, la lumière est éteinte. Vous pouvez discuter ce soir d'une stratégie commune, mais à partir de demain vous serez isolés et ne pourrez plus communiquer. Chaque jour, je tirerai au hasard de manière uniforme l'un d'entre vous, qui passera la journée dans la pièce. À tout moment durant sa visite, il pourra déclarer : *Nous avons tous visité la pièce*. S'il a raison, vous êtes libres. Sinon, je vous reboote les cerveaux ! »

Hypothèses implicites :

- La lumière est éteinte au départ.
- L'interrupteur est binaire (allumé/éteint).
- La lumière est invisible depuis l'extérieur de la pièce, et il n'y a aucun autre moyen de communication entre les étudiants.

4a. Énoncez précisément ce que doit garantir une stratégie réussie.

Une stratégie correcte doit vérifier :

Sûreté. Si un étudiant déclare « Nous avons tous visité la pièce », alors tous les étudiants l'ont effectivement visitée.

Vivacité. Avec probabilité 1, un étudiant finira par pouvoir faire cette déclaration.

4b. Proposez une stratégie et justifiez brièvement en quoi elle satisfait l'exigence précédente.

Une stratégie possible consiste à désigner un étudiant comme *délégué* :

- Chaque étudiant non délégué allume la lumière la première fois qu'il entre dans la pièce et la trouve éteinte, puis ne la rallume plus jamais.
- Chaque fois que le délégué entre dans la pièce et voit la lumière allumée, il l'éteint et incrémente un compteur local.
- Lorsque son compteur atteint $n - 1$, il déclare que tous les étudiants ont visité la pièce.

Cette stratégie garantit la sûreté (le délégué ne peut compter qu'un allumage par étudiant) et la vivacité presque sûrement (le tirage aléatoire finit par sélectionner chaque étudiant au moins une fois alors que la lumière est éteinte).

4c. Reprenez la question précédente, sans l'hypothèse sur l'état initial de la lumière.

Si l'état initial de la lumière est inconnu, le protocole peut être adapté en demandant au délégué d'ignorer le premier allumage observé (pour éliminer l'incertitude sur l'état de départ).

4d. Reprenez les deux premières questions, sans l'hypothèse que le tirage est uniforme. À la place, on suppose que si personne ne déclare que tout le monde a visité la pièce, chaque étudiant est sélectionné une infinité de fois (pas nécessairement à intervalles bornés).

1. Inspiration : M. Herlihy & N. Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann (2008)

Si le tirage n'est pas uniforme mais seulement équitable (chaque étudiant est choisi une infinité de fois, sans borne sur l'intervalle entre deux visites), la stratégie reste correcte : la sûreté n'est pas affectée, et la vivacité est renforcée et devient déterministe : un étudiant finira par pouvoir faire cette déclaration.

Séance 2

Synchronisation par verrous

Exercice 2.1 (Le dîner des philosophes²).

Des philosophes sont assis autour d'une table ronde. Devant chaque philosophe se trouve un bol de riz. Entre chaque paire de bols voisins se trouve une baguette. Ces philosophes occupent la majorité de leur temps à réfléchir, mais entre deux périodes d'intense réflexion, un philosophe a besoin de se nourrir. Pour manger, il doit commencer par prendre sa baguette de gauche et sa baguette de droite. Il peut alors manger un peu de riz. Une fois rassasié, il repose les deux baguettes. Faisons l'hypothèse que chaque philosophe mange pendant un temps fini et que le bol contient tellement de riz qu'il en restera toujours.

- 1a. En utilisant un théorème du cours, proposez une solution deadlock-free.
- 1b. Cette solution est-elle starvation-free ?

2. Inspiration : E. W. Dijkstra, *Hierarchical ordering of sequential processes*. Acta informatica (1971)

Séance 2

Synchronisation par verrous

Exercice 2.2 (Les comptes en banque).

Une banque vous demande d'implémenter son nouveau système d'information. Chaque compte de ses clients est représenté par un numéro de compte entier et possède un certain solde entier en centimes d'euros. Au centre de son système réparti, les comptes de ses clients sont gérés par une classe `Banque` possédant les deux méthodes suivantes.

- La méthode `int solde(int compte)` retourne le solde du compte passé en paramètre.
- La méthode `boolean transfere(int montant, int de, int a)` vérifie que les fonds sont disponibles sur le compte `de` (sinon, elle s'arrête en retournant `false`), puis elle soustrait `montant` du compte `de` et l'ajoute au compte `a` de manière atomique, puis retourne `true`.

On supposera que l'ensemble des comptes est représenté par un tableau de taille fixe d'entiers positifs ou nuls. Lors de l'initialisation, on dotera chaque compte d'un montant fixe.

On souhaite une implémentation de `Banque` satisfaisant la contrainte suivante :

« Si deux transferts ne concernent pas les mêmes comptes, on ne veut pas empêcher le parallélisme entre les deux appels à `transfere`. »

- 2a.** Comment appelle-t-on la stratégie à mettre en place pour satisfaire la contrainte ?
- 2b.** Proposez une implémentation de `Banque` satisfaisant la contrainte.

Séance 2

Synchronisation par verrous

Exercice 2.3 (Confection de tartes).

On veut modéliser le fonctionnement d'un atelier de pâtisserie. Pour préparer une tarte aux fraises, les pâtissiers doivent effectuer plusieurs étapes modélisées par des appels de fonctions comme ceci :

- `Fond preparerFond()` : prépare et étale un fond de tarte (20 minutes);
- `Pate cuire(Fond fond)` : cuit un fond de tarte (10 minutes);
- `Creme preparerCreme()` : prépare une crème pâtissière (10 minutes);
- `Tarte monter(Pate pate, Creme creme)` : étale la crème sur la pâte, et dispose les fruits (15 minutes).

3a. Proposez un programme Java parallélisant la création de tartes. Chaque thread représentera un pâtissier spécialisé sur une, et une seule, tâche. Vous embaucherez autant de pâtissier que nécessaire pour vous assurer qu'aucun d'entre eux ne reste inactif.

Séance 2 — pour aller plus loin

Synchronisation par verrous

Exercice 2.4 (Arbre binaire de recherche).

On cherche à implémenter un arbre binaire de recherche (ABR) en se basant sur le code suivant. Identifier les problèmes de concurrence qui peuvent se poser, puis proposez un moyen de les résoudre.

```
public class ABR {
    private Node root = new Node();

    public boolean contains(int v) {
        return root.get(v).isSet;
    }

    public void insert(int v) {
        Node n = root.get(v);
        if(!n.isSet) n.setValue(v);
    }

    public void delete(int v) {
        Node n = root.get(v);
        if(!n.isSet) return;
        if(!n.smaller.isSet) n.copyNode(n.greater);
        else {
            Node predecessor = n.smaller;
            while(predecessor.greater.isSet)
                predecessor = predecessor.greater;
            n.value = predecessor.value;
            predecessor.copyNode(predecessor.smaller);
        }
    }
}

class Node {
    Node smaller = null;
    boolean isSet = false;
    int value;
    Node greater = null;

    void copyNode(Node n) {
        isSet = n.isSet; value = n.value; smaller = n.smaller; greater = n.greater;
    }
    void setValue(int v) {
        isSet = true; value = v; smaller = new Node(); greater = new Node();
    }

    Node get(int v) {
        if(!isSet || v == value) return this;
        if (v < value) return smaller.get(v);
        return greater.get(v);
    }
}
```

Séance 3

Moniteurs de synchronisation

Exercice 3.1 (La pile bloquante).

Une pile bloquante bornée, dont la capacité est passée en paramètre au constructeur, possède deux opérations :

- `void push(T)` attend qu'il y ait de la place dans la pile puis ajoute un élément en tête de la pile ;
- `T pop()` attend que la pile ne soit pas vide puis supprime et retourne l'élément en tête de la pile.

Implémentez une pile bloquante bornée en Java.

Séance 3

Moniteurs de synchronisation

Exercice 3.2 (Les toilettes unisexes³).

Les toilettes unisexes peuvent être utilisées par les hommes et par les femmes, sous les contraintes suivantes :

1. il ne doit jamais y avoir en même temps des hommes et des femmes aux toilettes ;
2. il ne doit jamais y avoir plus de trois personnes simultanément aux toilettes.

Un programme modélisant le problème est disponible sous Madoc. Le comportement des personnes qui accèdent aux toilettes est déjà modélisé. Les toilettes doivent implémenter l'interface `Bathroom`, qui contient une méthode `void enter(boolean isMale)` et une méthode `void leave(boolean isMale)`.

2a. Proposez une solution deadlock-free.

2b. Modifiez votre solution pour la rendre starvation-free.

Indice : des humains bien élevés organiseraient probablement une file d'attente.

3. Inspiration : A. B. Downey. *The Little Book of Semaphores*. Green Tea Press (2008)

Séance 3

Moniteurs de synchronisation

Exercice 3.3 (Implémentation d'un thread pool).

On rappelle les interfaces (partielles) des types permettant de gérer les thread pools en Java.

```
interface Callable<T> {  
    T call();  
}  
interface Future<T> {  
    boolean isDone();  
    T get(); // Met le thread en attente jusqu'à la fin de la tâche  
}  
interface ThreadPool { Future<T> submit(Callable<T> task); }
```

- 3a.** Proposez une implémentation de `Future` et de `ThreadPool`, permettant de gérer un thread pool de taille fixe (le nombre de threads sera passé en paramètre au constructeur).

On souhaite modifier la signature de `call()` et de `get()` en ajoutant `throws Exception`, pour autoriser les tâches à lever des exceptions. Si une tâche lève une exception, elle doit pouvoir être gérée lors de la récupération du résultat.

- 3b.** Modifiez votre code pour prendre en compte les exceptions.

Séance 3 — pour aller plus loin

Moniteurs de synchronisation

Exercice 3.4 (Barrière de synchronisation).

Le but de cet exercice est d'implémenter une barrière de synchronisation permettant de s'assurer qu'un certain nombre de threads n aient tous terminé une première phase avant de passer à la seconde. Plus précisément, la barrière contient un compteur initialisé avec le paramètre n de son constructeur, et expose une méthode `void await()`. Quand un thread appelle la méthode `await`, le compteur est décrémenté et le thread appelant est mis en attente tant que le compteur est strictement positif. Autrement dit, les n premiers threads appelant la méthode `await` doivent attendre qu'au moins n threads l'aient appelée. Quand n threads ou plus ont appelé la méthode `await`, tous les threads sont libérés et peuvent continuer leur exécution.

- 4a.** Donnez une implémentation en Java d'une barrière de synchronisation, sous la forme d'un moniteur.

Séance 3 — pour aller plus loin

Moniteurs de synchronisation

Exercice 3.5 (Bande de sauvages⁴).

Une tribu de sauvages mange son dîner dans un pot commun qui peut contenir M parts de potage. Quand un sauvage veut manger, il se sert au pot, sauf s'il est vide. Quand le pot est vide, les sauvages réveillent le cuisinier et attendent qu'il remplisse le pot.

Programmer le comportement des sauvages et du cuisinier. Les contraintes sont les suivantes :

- Les sauvages ne peuvent pas se servir dans un pot vide ;
- Le cuisinier ne doit remplir le pot que quand il est vide ;
- Discutez les propriétés de vivacité obtenues.

4. Inspiration : G. R. Andrews. *Foundations of Multithreaded, Parallel, and Distributed Programming*. Addison-Wesley (2000)

Séance 3 — pour aller plus loin

Moniteurs de synchronisation

Exercice 3.6 (Le mot-clé `select` en go).

En Go, un canal (*channel*) est une construction du langage, proche d'une `BlockingQueue` en Java, qui permet de transférer des informations entre goroutines et qui peut bloquer jusqu'à ce qu'un élément soit disponible (lecture) ou qu'une place se libère (écriture). L'instruction `select` permet d'attendre plusieurs opérations sur des canaux et de débloquent dès que l'une d'elles est prête. Par exemple, le programme suivant affiche le premier message reçu, soit depuis `channel1`, soit depuis `channel2`.

```
select {  
  case msg1 := <-channel1: fmt.Println(msg1)  
  case msg2 := <-channel2: fmt.Println(msg2)  
}
```

- 6a.** Il n'existe pas d'équivalent direct en Java. Concevez une abstraction qui permette d'attendre sur plusieurs files bloquantes (type `BlockingQueue`) et de reprendre dès que l'une d'elles est prête.

Séance 4

Le modèle mémoire de Java

Exercice 4.1 (Volatilité).

Les trois codes suivants s'exécutent en parallèle, avec initialement, $x = y = z = 0$.

```
/****** Thread 1 *****/  
x = 1;  
y = 1;  
  
/****** Thread 2 *****/  
if (y == 1)  
  z = 1;  
  
/****** Thread 3 *****/  
if (z == 1)  
  System.out.println(x);
```

- 1a. Quelles sont les sorties possibles si x , y et z sont déclarés `volatile` ?
- 1b. Quelles variables faut-il déclarer `volatile` pour éliminer tout data race ?

Séance 4

Le modèle mémoire de Java

Exercice 4.2 (Singleton).

On rappelle le code séquentiel du patron de conception *Singleton*.

```
class Singleton {  
    private static Singleton instance = null;  
    private Singleton() {}  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

- 2a. Donnez une exécution avec deux threads où deux instances sont créées.
- 2b. Corrigez le programme. Il faudra réduire l'usage des verrous au maximum.

Séance 4

Le modèle mémoire de Java

Exercice 4.3 (Identification de *data races*).

On examine le programme Java suivant. Les trois variables partagées sont initialisées à `x = 0` et `flag = true`. On suppose qu'aucun autre thread n'intervient et que les réveils intempestifs possibles de `wait()` sont correctement gérés par la boucle `while`. On raisonne sous l'hypothèse de *cohérence séquentielle* pour les questions 1–3.

```
1  final class Program {
2      static final Object o = new Object();
3      static int x = 0;
4      static boolean flag = true;
5
6      public static void main(String[] args) throws Exception {
7          Thread t = new Thread(() -> {
8              x = 1;
9              synchronized (o) {
10                 flag = false;
11                 o.notify();
12             }
13             x = 2;
14         });
15
16         synchronized (o) {
17             t.start();
18             while (flag) {
19                 o.wait();
20             }
21             int a = x;
22         }
23         int b = x;
24     }
25 }
```

- 3a. Donnez la machine à états du programme.
- 3b. Déduisez-en toutes les exécutions séquentiellement cohérentes. Quelles sont les valeurs possibles de *a* et *b*?
- 3c. Identifiez toutes les *data races* présentes dans le programme.
- 3d. Donnez deux corrections *minimales* et *distinctes* qui éliminent toutes les *data races* :
 - en ne modifiant que des qualificatifs (ex. `volatile`) dans les déclarations;
 - en ne modifiant que la synchronisation (ex. blocs `synchronized`).
- 3e. Pour chacune des deux corrections, quelles sont alors les valeurs possibles de *a* et *b*?

Séance 5

Algorithmes d'exclusion mutuelle

Exercice 5.1 (L'algorithme de filtre⁵).

On considère l'algorithme suivant, pour 3 threads d'identifiants 0, 1 et 2.

```
class PetersonFor3 {  
    private final AtomicIntegerArray entering = new AtomicIntegerArray(3);  
    private volatile int lastArrived;  
    public void lock() {  
        int i = ThreadID.get();  
        entering.set(i, 1);  
        lastArrived = i;  
        for(int j = 0; j < 3; j++) {  
            while(j != i && entering.get(j) == 1 && lastArrived == i);  
        }  
    }  
    public void unlock() {  
        int i = ThreadID.get();  
        entering.set(i, 0);  
    }  
}
```

- 1a. Montrez qu'au plus deux threads peuvent accéder en même temps en section critique.
- 1b. Combinez cet algorithme à l'algorithme de Peterson pour concevoir un verrou entre trois threads.
- 1c. Généralisez à n threads.

5. Algorithme : J. L. M. Kessels. *On the existence of wait-free protocols for mutual exclusion in hard real-time environments*. IPL (1982)

Séance 5

Algorithmes d'exclusion mutuelle

Exercice 5.2 (Verrou réentrant).

On rappelle qu'un verrou est réentrant si un thread qui a déjà pris un verrou peut ré-entrer en section critique, ce qui signifie qu'il ne sera pas bloqué s'il appelle `lock()` à nouveau. Pour relâcher le verrou, un thread doit appeler `unlock()` autant de fois qu'il a appelé `lock()`.

- 2a.** On se donne une implémentation non réentrante des verrous. La transformer en une implémentation réentrante.

Séance 5

Algorithmes d'exclusion mutuelle

Exercice 5.3 (L'instruction `getAndIncrement`⁶).

On rappelle que la méthode `AtomicInteger.getAndIncrement()` incrémente le contenu de son instance, et retourne la valeur écrasée de manière atomique.

- 3a.** Proposez une implémentation d'un verrou *starvation-free* en utilisant l'instruction spéciale `getAndIncrement` et capable de supporter un nombre de threads inconnu lors de sa construction.

6. Algorithme : J. M. Mellor-Crummey, M. L. Scott. *Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors*. TOCS (1991)

Séance 5 — pour aller plus loin

Algorithmes d'exclusion mutuelle

Exercice 5.4 (Verrou à *lecture/écriture*).

On rappelle qu'un verrou à *lecture/écriture* peut être pris soit en mode lecture (partagé entre tous les lecteurs), soit en mode écriture (exclusif à l'écrivain). Plusieurs lecteurs peuvent exécuter leur section critique de manière concurrente, mais un écrivain doit toujours être seul en section critique.

- 4a.** En vous inspirant de l'algorithme de la boulangerie, proposez une implémentation des verrous à lectures et écritures.